

HOLISTIC BUILD DOCUMENT **APEX-1**  
Autonomous High-Speed Grand Prix Rover

---

**Category:** Robo Grand Prix

**Team Name:** Team Apex

**Academic Institution:** Tshwane University of Technology

**Engineering Personnel**

**Mathurin Moundzika-Kibamba** — Systems Architect; Electrical Distribution & Power Systems Lead;  
*Mechanical CAD & Fabrication Lead; Assembly & Hardware Implementation Co-Lead*

**Ayanda Tshabangu** — Software Architecture Lead; Embedded Firmware &  
*Low-Level Driver Development Lead; OpenCV Implementation; Hardware Implementation Co-Lead*

# Table of Contents

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| Table of Contents .....                                                                | 2  |
| 2. Context & Problem Statement.....                                                    | 2  |
| 2.1 Competition Constraints (Robo Grand Prix) .....                                    | 2  |
| 2.2 Engineering Challenge & Failure Modes.....                                         | 2  |
| 2.3 The APEX-1 Approach .....                                                          | 3  |
| 3. Solution Overview.....                                                              | 3  |
| 4. Bill of Materials (BOM) Summary .....                                               | 4  |
| 5. Mechanical Design Section (Robo Grand Prix Rigour) .....                            | 5  |
| 5.1 Fabrication Methods.....                                                           | 5  |
| 5.2 Centre of Gravity (CoG) & Mass Distribution.....                                   | 5  |
| 5.3 Drive Kinematics & Locomotion .....                                                | 6  |
| 5.4 Sensor Enclosures & Protective Integration .....                                   | 7  |
| 6. Electronic Design Section (Robo Grand Prix Rigour).....                             | 8  |
| 6.1 Power Distribution Network (PDN) & Thermal Strategy .....                          | 8  |
| 6.2 Mandatory Emergency Stop (E-Stop) Integration.....                                 | 9  |
| 6.3 Inter-Integrated Circuit (I <sup>2</sup> C) Bus Topology & Address Resolution..... | 10 |
| 7. Programming & Framework Design Section (Robo Grand Prix Rigour) .....               | 12 |
| 7.1 Distributed Software Architecture.....                                             | 12 |
| 7.2 Low-Level Control Loops & Sensor Fusion (STM32 / C++) .....                        | 13 |
| 7.3 High-Level Vision & Path Generation (Custom OpenCV Pipeline) .....                 | 14 |
| 7.4 Core Firmware Snippet (Dynamic Corner-Speed Control).....                          | 15 |
| 8. Verification, Testing Protocols, & Race Preparation.....                            | 16 |
| 8.1 Rigorous Subsystem Testing Protocol .....                                          | 16 |
| 8.2 Summary Conclusion .....                                                           | 16 |

## 2. Context & Problem Statement

### 2.1 Competition Constraints (Robo Grand Prix)

The Robo Grand Prix rules state that vehicles must navigate the track under strictly autonomous operation, with no human intervention allowed.

- **Mass Limit:** 5 kg
- **Dimensional Limits:** 50 cm × 50 cm
- **Safety & Emergency Cutoff:** a mandatory, clearly accessible Emergency Stop button capable of instantly cutting off all power to the vehicle's circuit.
- **No Tethering:** no off-board compute or telemetry overrides are allowed during competitive runs.

### 2.2 Engineering Challenge & Failure Modes

Traditional vehicles fail due to three main engineering bottlenecks:

1. **Compute Starvation:** running vision algorithms and live PID loops on a single microcontroller, or a single-board computer, leads to control lag — resulting in overshoot at high speeds.

2. **Electrical Brownouts & Transients:** DC motors drawing high current under sudden directional reversals cause severe voltage drops, resetting onboard logic controllers mid-race.
3. **Sensor Blind Spots:** relying only on cameras introduces delays in proximity detection, while relying only on short-range IR sensors creates lookahead blindness at high speed.

## 2.3 The APEX-1 Approach

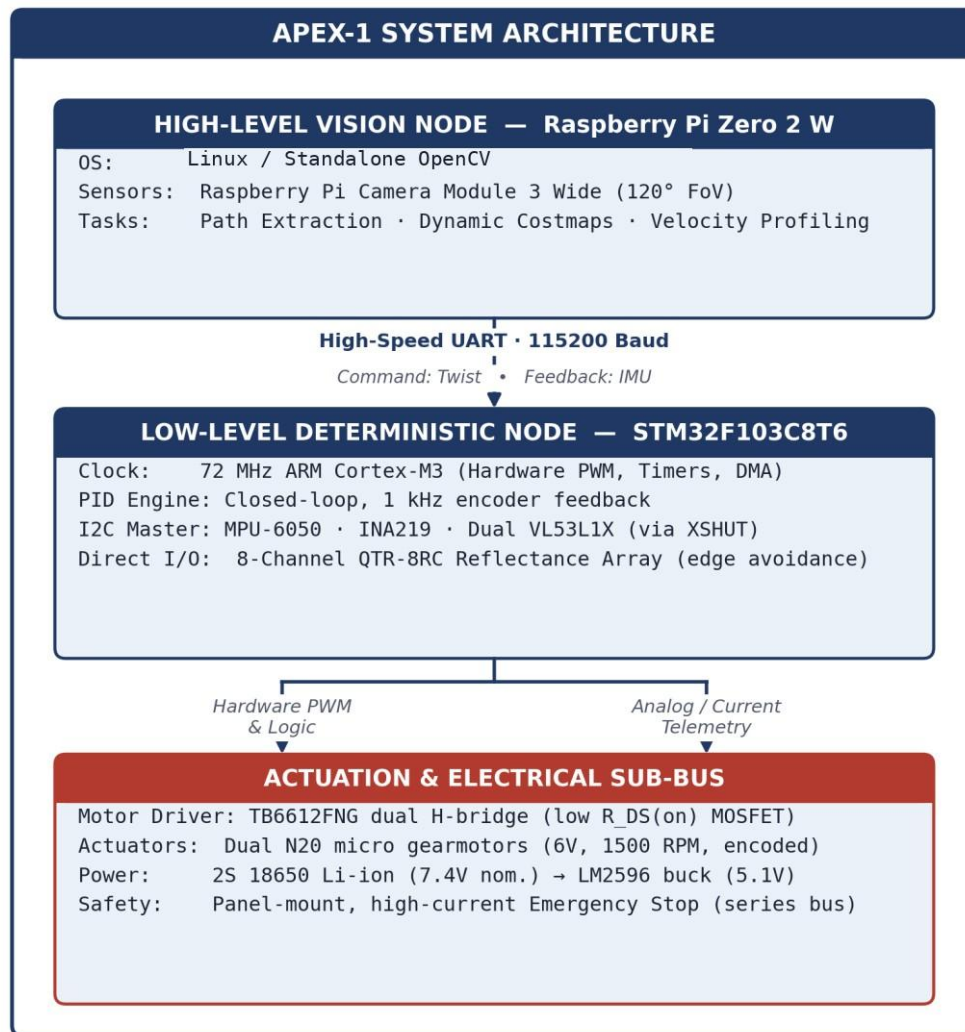
APEX-1 addresses these constraints through a distributed dual-processing architecture, a low centre of gravity, a 28 cm × 18 cm 3D-printed chassis, a segregated power bus, and a fused three-tier perception array. By keeping the dry weight under 1 kg, APEX-1 maximises power-to-weight ratio to ensure fast braking and sharp steering response, well within the competition limits.

## 3. Solution Overview

---

APEX-1 is an agile, differential-drive autonomous racing vehicle engineered for competitive track environments. The platform splits computational workloads between deterministic low-level control and high-level edge intelligence.

- **Deterministic Low-Level Node (STM32 Blue Pill):** executes live motor control, closed encoder PID calculations, high-speed Time-of-Flight distance polling, reflectance line tracking, and current monitoring.
- **High-Level Edge Vision Node (Raspberry Pi Zero 2 W):** runs an optimised OpenCV environment to process wide-angle camera feeds, track lane vectors, and execute live local path planning.
- **Physical Architecture:** a highly rigid, 3D-printed chassis using lightweight PLA with internal wire ducting, isolating electronic logic from high-vibration gear motors and heat-generating regulators.



**Figure 1.** APEX-1 system architecture — high-level vision node, low-level deterministic node, and the actuation / electrical sub-bus.

## 4. Bill of Materials (BOM) Summary

The full implementation cost is R3,545.39. The implementation prioritises South African component suppliers to ensure rapid local delivery and supply-chain resilience. All prices reflect real-world, verified rates including VAT (excluding final localised shipping fees).

Below is a summary of the primary architectural hardware used in the APEX-1 build:

- **Primary Edge Compute:** Raspberry Pi Zero 2 W — [View Product](#)
- **Real-Time Microcontroller:** STM32F103C8T6 Blue Pill — [View Product](#)
- **Vision Hardware:** Raspberry Pi Camera Module 3 Wide — [View Product](#)
- **Actuation:** Dual N20 Micro Gearmotors (6V, 1500 RPM) — [View Product](#)
- **Ranging & Telemetry:** Dual VL53L1X ToF Sensors & QTR-8RC IR Array — [View Product](#) · [View Product](#)

For the complete, itemised financial breakdown, please open the accompanying **Bill\_of\_Materials.xlsx** file included in this Folder C submission.

## 5. Mechanical Design Section (Robo Grand Prix Rigour)

### 5.1 Fabrication Methods

The APEX-1 mechanical architecture utilises a multi-tier structural design fabricated via Fused Deposition Modelling (FDM) 3D printing using eSun PLA Basic. The fabrication strategy was strictly guided by the need to balance technical strength, physical stability, and economic feasibility, ensuring the build is structurally robust without relying on needlessly expensive exotic materials.

- **Material Justification & Economics:** PLA was selected because it provides an optimal strength-to-weight ratio. It keeps the chassis lightweight enough for rapid acceleration while maintaining enough rigidity to prevent chassis flex under high motor torque. Economically, it hits the perfect sweet spot — costing significantly less than carbon-infused filaments while outperforming cheaper, brittle plastics.
- **Variable Slicing Parameters:** to optimise mass without compromising durability, variable infill densities are applied within the slicer. The primary shell utilises a 15%–20% Gyroid or TriHexagonal infill pattern for excellent isotropic shear resistance, with localised density increased to 20%–25% at all critical mounting points to prevent thread stripping and fracture.
- **Kinematic Optimisation & Thermals:** the chassis geometry is engineered to concentrate the vehicle's weight as close to the ground as possible, maximising tyre traction and stabilising the robot during high-speed cornering — paired with smaller-diameter drive wheels that tighten the turning radius.
- **Thermal Management:** strategic ventilation slots are designed into the upper chassis cowl, particularly around the main power button and electronics mezzanine, ensuring passive convective cooling during prolonged track runs.
- **Tolerances:** threaded mounting holes are designed with a nominal 3.2 mm diameter, sized precisely for M3 clearance and heat-set brass threaded inserts or direct nylon hardware through-bolting.

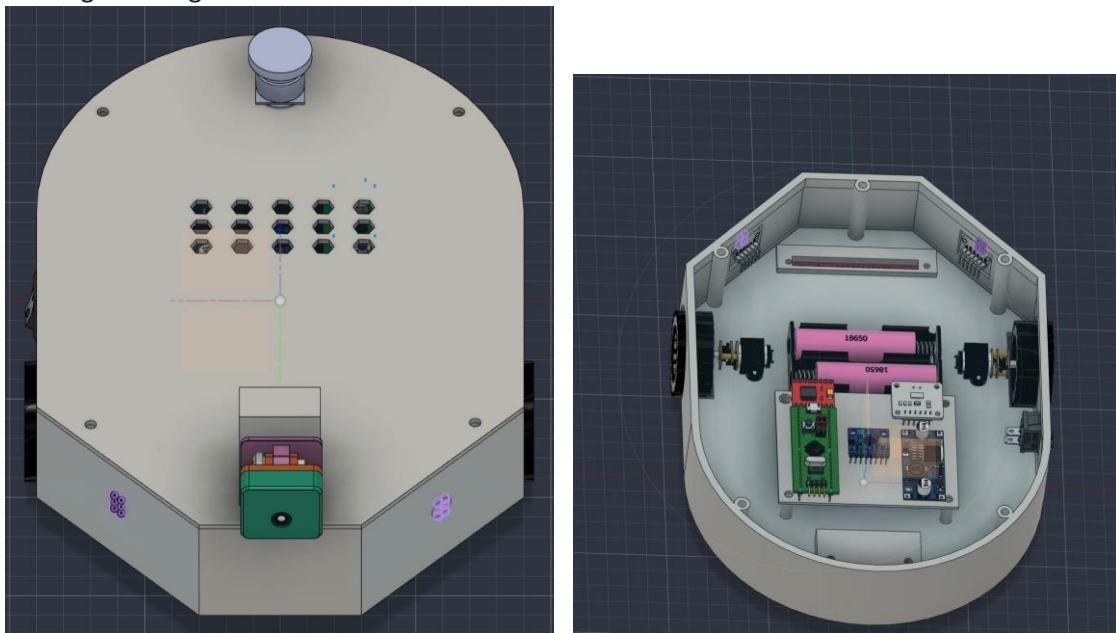


Figure 2. 3D CAD isometric assembly (left) and internal exploded view (right).

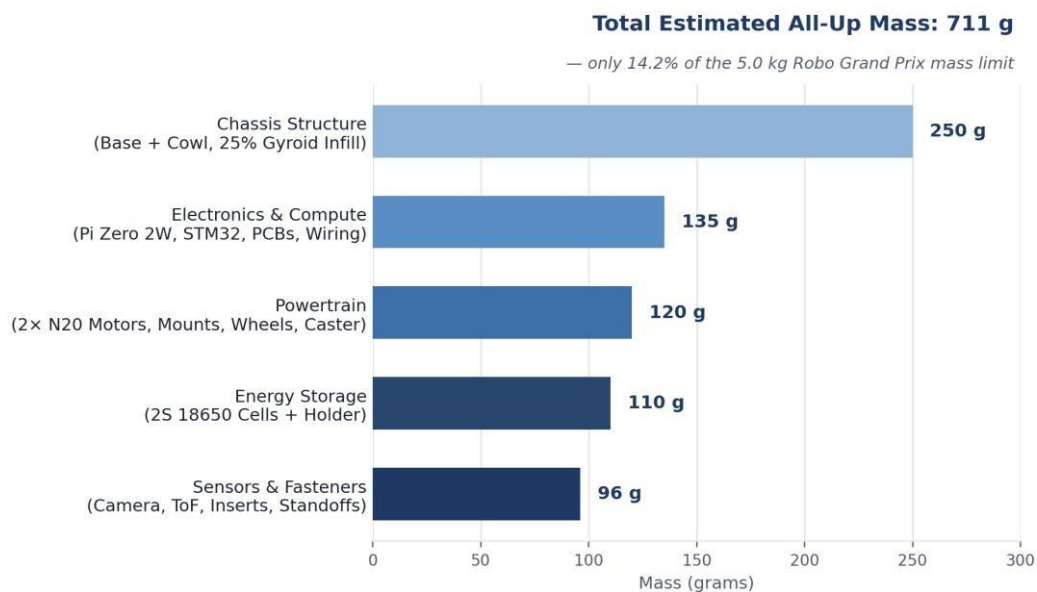
### 5.2 Centre of Gravity (CoG) & Mass Distribution

To mitigate the risk of traction loss during rapid cornering and prevent forward tipping under dynamic braking, the mass distribution is concentrated along the vehicle's bottom centreline:

- **Battery Placement:** the heaviest discrete payload item — the 2× 18650 cell array (≈ 95 g) — is slung in a recessed pocket directly between the driving wheels on the floor of the lower chassis tub, establishing an ultra-low centre of mass ( $h_{eom} \approx 14$  mm above the track surface).
- **Electronics Mezzanine:** the dual-sided perfboard carrying the STM32, buck converter, and sensor breakout boards is suspended 18 mm above the baseplate via M3 nylon standoffs, isolating structural road vibration from solder joints and eliminating electrical short risks.

### Mass Budget

| Mass Budget Item                                            | Mass         |
|-------------------------------------------------------------|--------------|
| Chassis Structure (Base + Cowl, 25% Gyroid Infill)          | 250 g        |
| Powertrain (2× N20 Motors, Mounts, Wheels, Ball Caster)     | 120 g        |
| Energy Storage (2S 18650 Cells + Holder)                    | 110 g        |
| Electronics & Compute (Pi Zero 2W, STM32, PCBs, Wiring)     | 135 g        |
| Sensors & Fasteners (Camera, ToF, Brass Inserts, Standoffs) | 96 g         |
| <b>Total Estimated All-Up Mass</b>                          | <b>711 g</b> |



**Figure 3.** Mass budget by subsystem — 711 g all-up, well within the 5 kg competition limit.

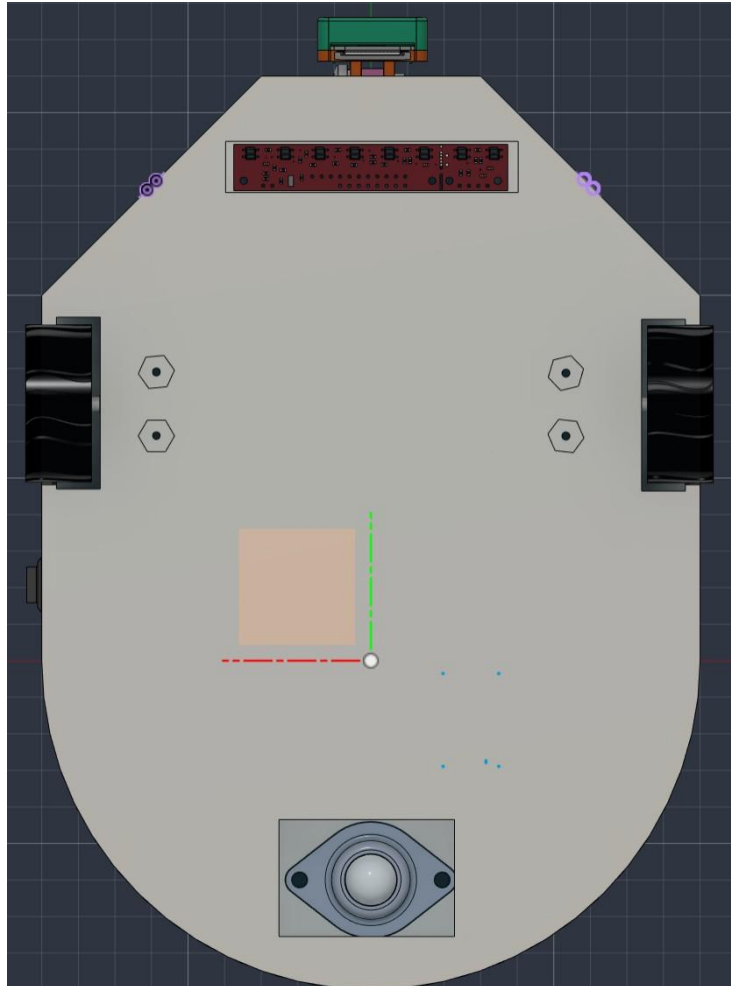
## 5.3 Drive Kinematics & Locomotion

APEX-1 employs a two-wheel differential-drive topology augmented by a smooth, omnidirectional front ball caster:

- **Drive Axle:** powered by two N20 micro metal gearmotors geared to 1500 RPM at 6V. At a 43 mm wheel diameter, the theoretical free-run linear surface speed is:

$$v_{max} = \pi \times d \times \frac{RPM}{60} = \pi \times 0.043 \text{ m} \times \frac{1500}{60} \approx 3.38 \text{ m/s}$$

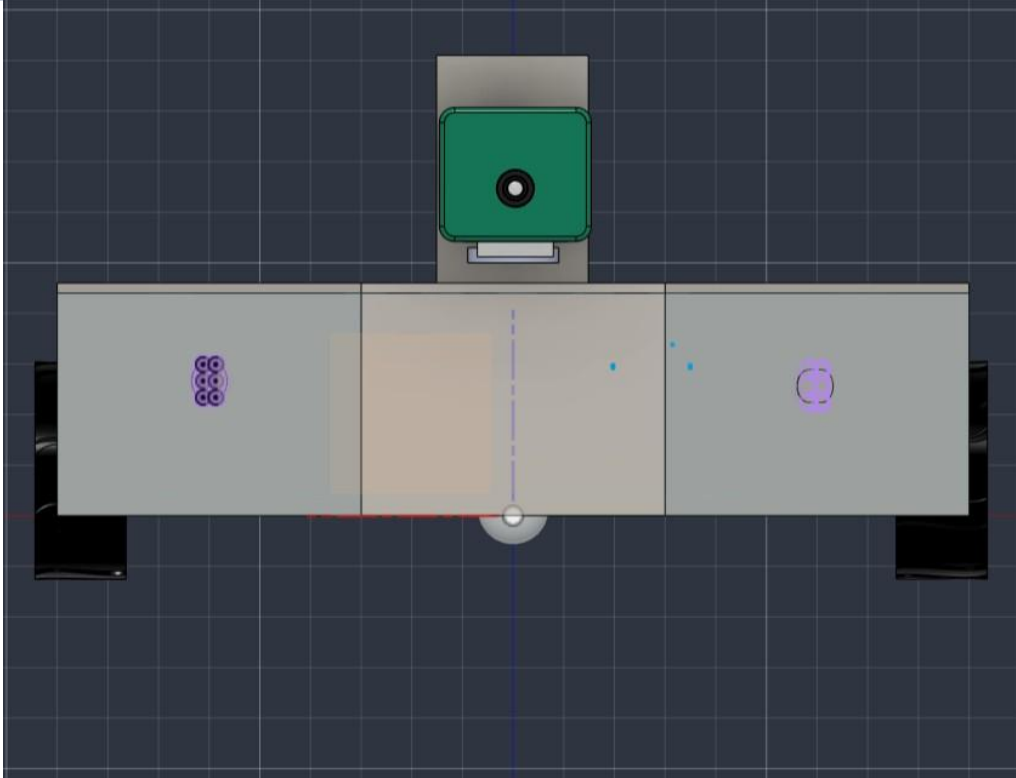
- Allowing for track friction, the 711 g chassis mass, and real-world voltage drops under load, the calculated functional operating speed sits safely in the 1.5–2.2 m/s envelope — a derived engineering estimate based on motor and wheel specifications, pending physical calibration.
- **Front Support:** a metal ball-caster wheel mounted forward provides low-friction, three-point planar contact, eliminating the scrub resistance typical of fixed skids during high-speed yaw changes.



*Figure 4. Chassis dimensioning, wheelbase, and centre-of-mass reference planes.*

## 5.4 Sensor Enclosures & Protective Integration

- **VL53L1X Laser Aperture Protection:** rather than leaving the sensitive SPAD receivers exposed to direct track impacts, the dual ToF sensors are mounted behind angled forward-facing protective bulkheads integrated directly into the 3D-printed chassis. Recessed openings provide an unobstructed 27° cone of view while acting as physical bumpers against walls.
- **Camera Cowl:** the Raspberry Pi Camera Module 3 Wide is secured in a vibration-damped front turret angled downward at 22° relative to the horizontal axis — capturing track boundaries up to 1.2 m ahead while maintaining visibility of the immediate 150 mm foreground.



*Figure 5. CAD rendering of VL53L1X sensor and camera placement.*

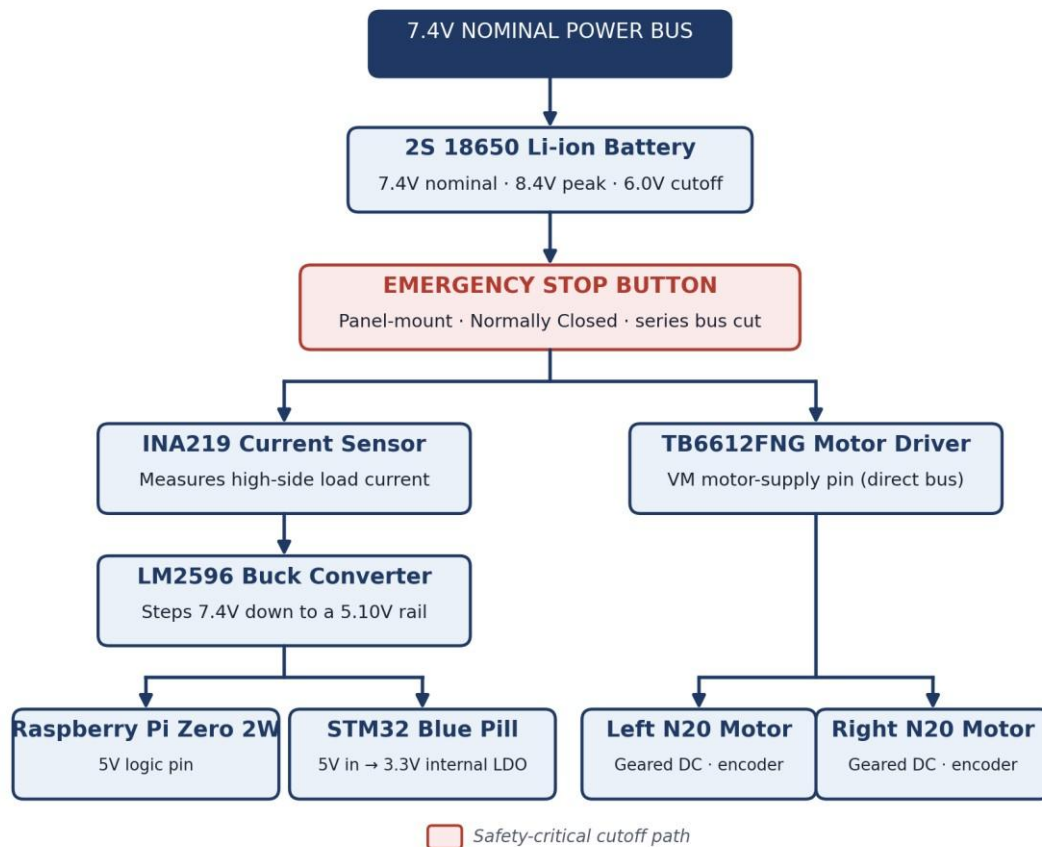
## 6. Electronic Design Section (Robo Grand Prix Rigour)

---

### 6.1 Power Distribution Network (PDN) & Thermal Strategy

The electrical architecture separates clean digital logic supply lines from high-noise inductive motor circuits.





**Figure 6.** Power distribution network — from the 7.4V nominal bus through the Emergency Stop to the logic and drive sub-systems.

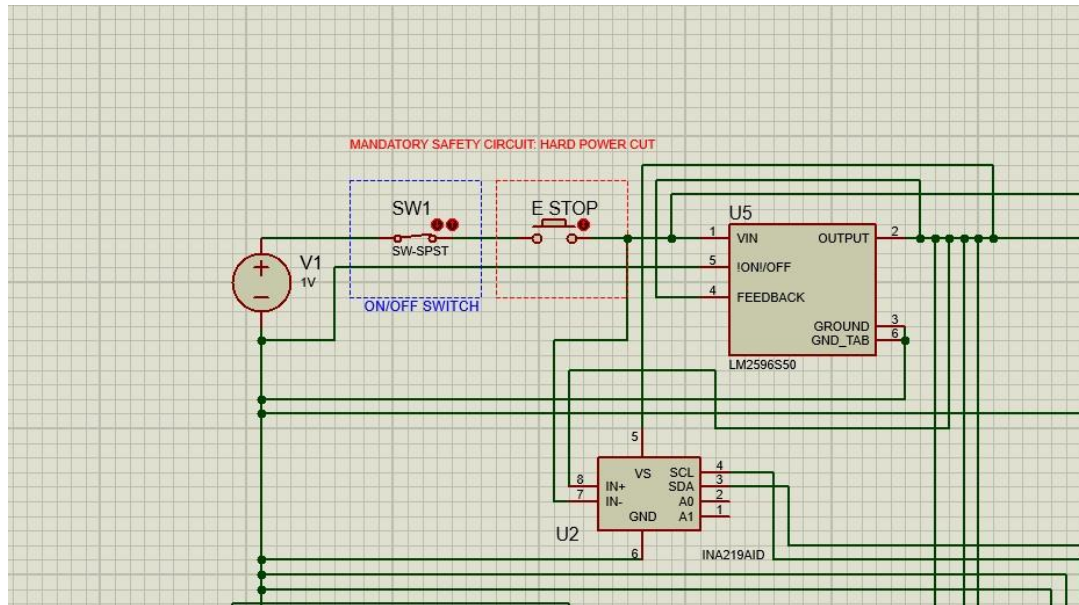
- **Energy Storage Bus:** two 18650 lithium-ion cells wired in series provide a 7.4V nominal bus voltage (8.4V peak charge, 6.0V safety low-voltage cutoff).
- **Switching Step-Down Regulation:** an LM2596 buck converter module steps the 7.4V down to a stable 5.10V logic rail capable of supplying up to 3.0A continuous. Large onboard electrolytic capacitors smooth transient ripple, preventing single-board-computer reboot cycles when motors start rapidly.
- **Logic Decoupling:** the Raspberry Pi Zero 2 W draws 5V directly from the regulated rail via GPIO header pins 2/4. The STM32 receives 5V on its 5V pin, using its internal low-dropout (LDO) regulator to cleanly generate its 3.3V internal core reference.
- **Direct Inductive Bus:** the TB6612FNG motor driver's VM pin is tied directly to the switched 7.4V high-current bus, bypassing the logic buck converter entirely to prevent back-EMF injection into sensitive digital traces.

## 6.2 Mandatory Emergency Stop (E-Stop) Integration

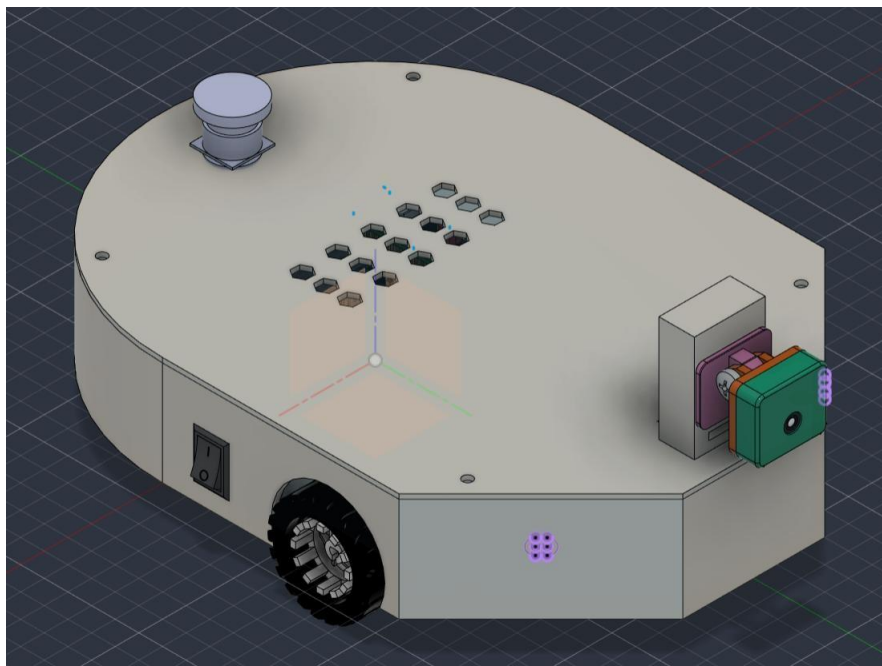
In compliance with competition safety mandates, a rugged 3-Pin Panel-Mount Emergency Stop push button is wired directly in series with the positive lead of the 18650 battery holder — before any regulator, capacitor, or driver connection:

- **Mechanism:** the switch operates on a normally closed (NC) high-amperage latching contact.
- **Actuation:** striking the prominent red mushroom button physically breaks the entire circuit, instantly de-energising both the drive motors and the computational logic, a direct hard power cut in series with the battery, independent of software or firmware state.

- **Mounting:** screwed securely through the reinforced rear apex of the upper chassis cowl, easily accessible from any vehicle orientation.



**Figure 7.** Safety-circuit detail — the hard power-cut path: battery → ON/OFF switch → E-Stop → regulator / current sensor, ahead of every other load.



**Figure 8.** CAD rendering of the physical Emergency Stop button and power switch placement on the chassis cowl.

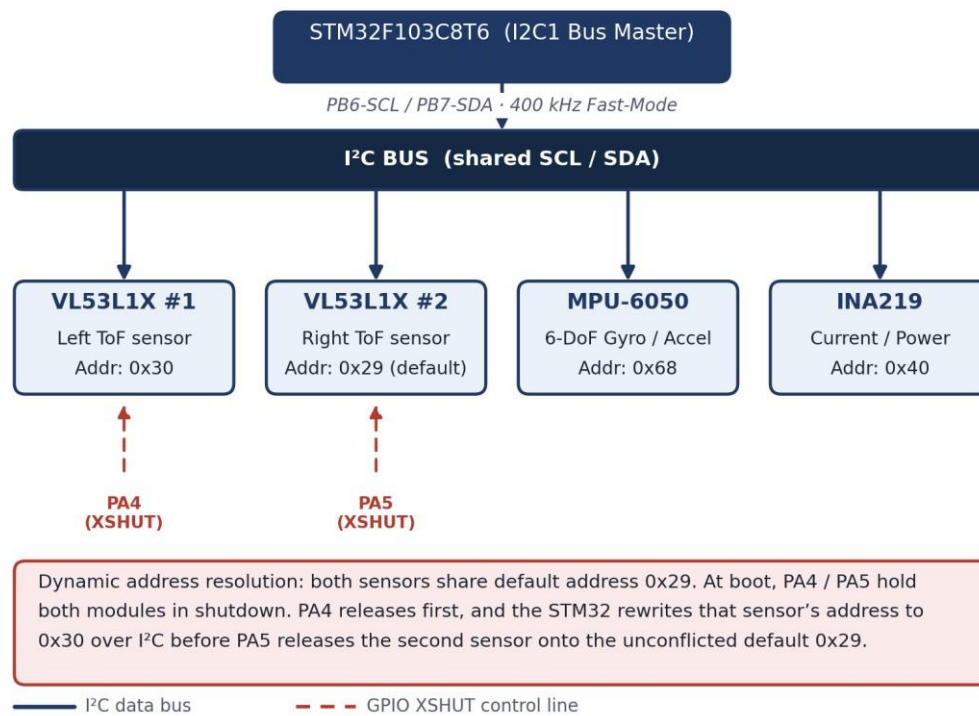
### 6.3 Inter-Integrated Circuit (I<sup>2</sup>C) Bus Topology & Address Resolution

The STM32 functions as the single I<sup>2</sup>C bus master (running on hardware I2C1: PB6-SCL, PB7-SDA) communicating at 400 kHz Fast-Mode.

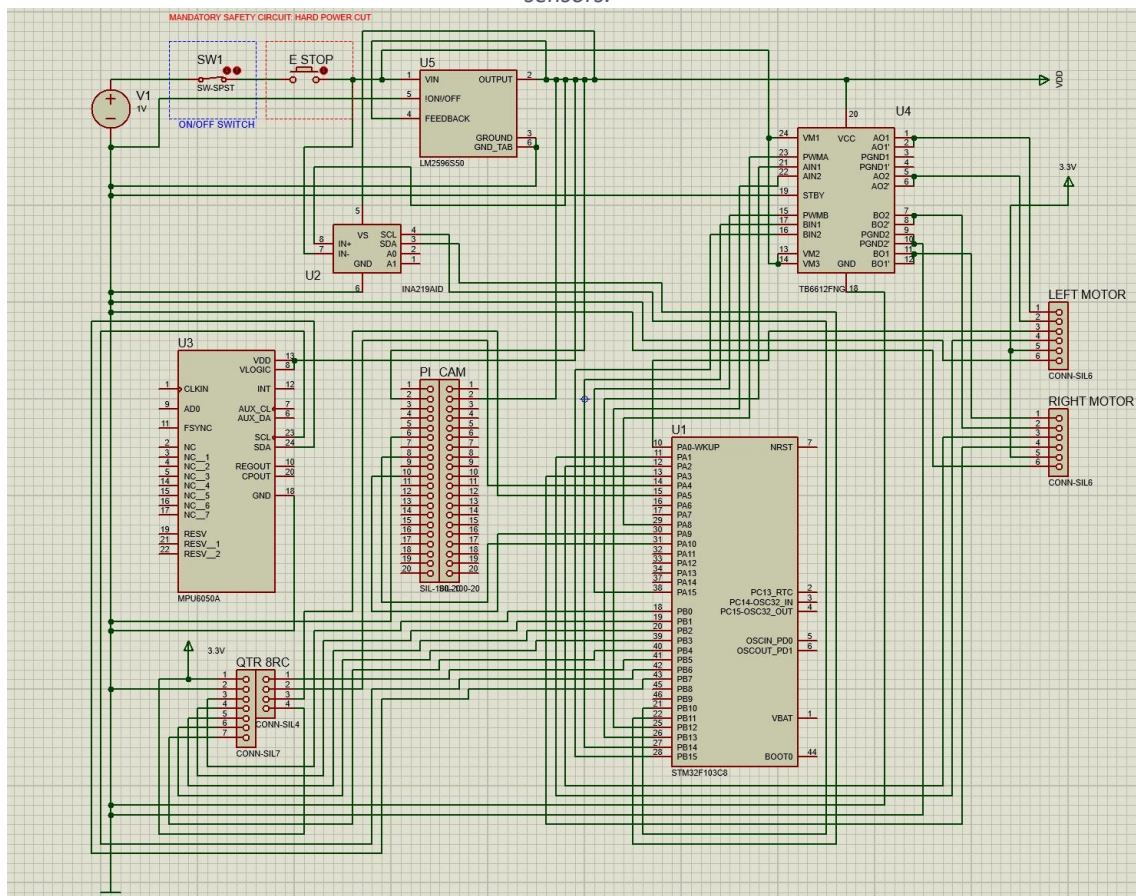
To resolve hardware conflicts caused by two identical VL53L1X sensors sharing the default address 0x29, a dynamic initialisation sequence is implemented using the hardware XSHUT (shutdown) pins:

1. At boot, both ToF sensors are pulled to ground via STM32 GPIO pins PA4 and PA5, forcing both modules into hardware shutdown.
2. PA4 is driven HIGH, booting ToF Sensor 1. The STM32 sends a software configuration command via I<sup>2</sup>C to rewrite Sensor 1's internal register, shifting its active address to 0x30.

3. PA5 is subsequently driven HIGH, booting ToF Sensor 2, which binds to the unconflicted default address 0x29.
4. The bus concurrently polls the INA219 current sensor at address 0x40 and the MPU-6050 IMU at address 0x68.



**Figure 9.** I²C bus topology and the dynamic XSHUT-based address-resolution sequence for the dual VL53L1X sensors.



**Figure 10.** APEX-1 circuit schematic — full power, safety, and signal wiring.

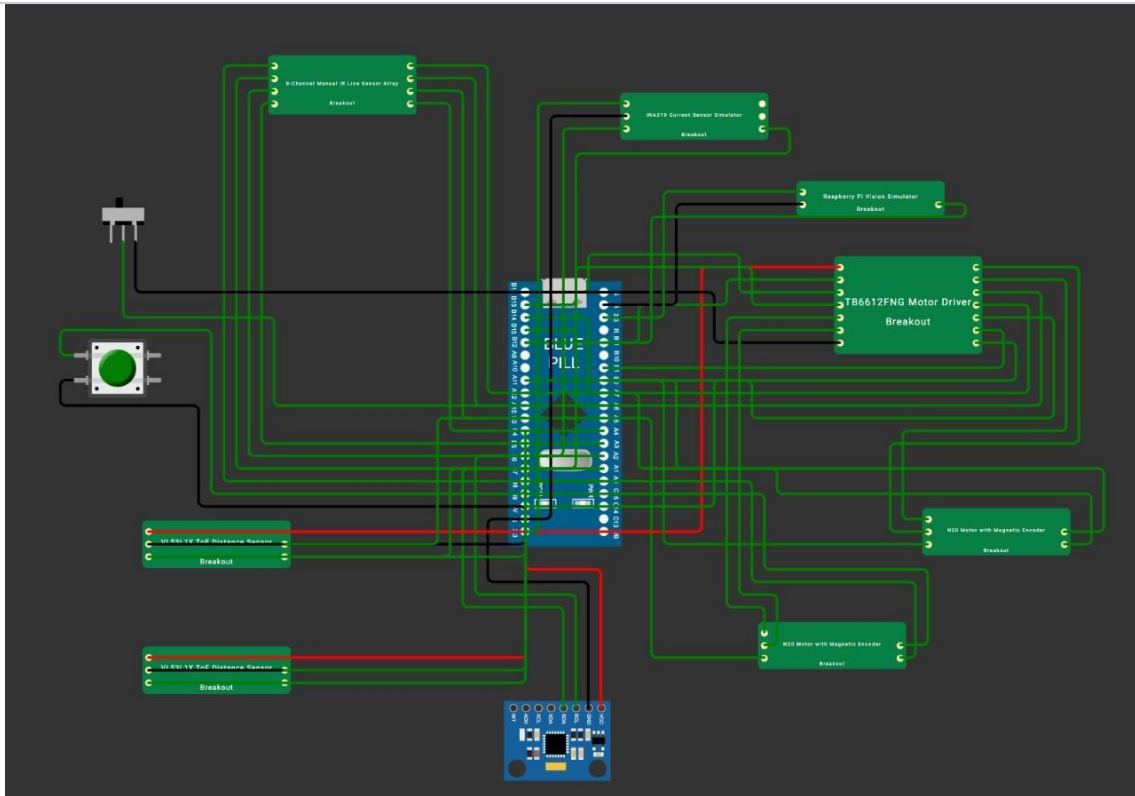


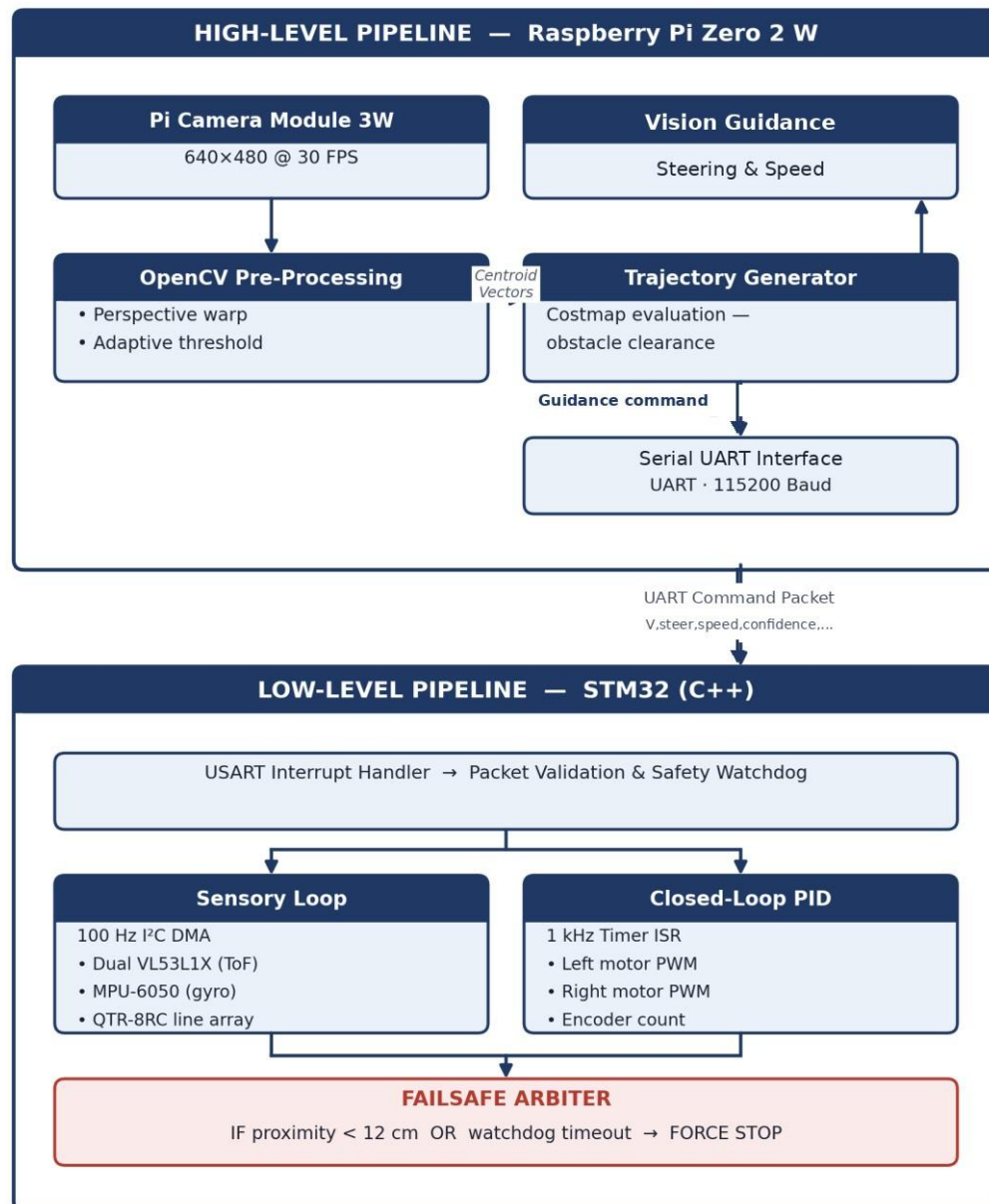
Figure 11. Physical wiring and breakout-board layout, as prototyped.

## 7. Programming & Framework Design Section (Robo Grand Prix Rigour)

### 7.1 Distributed Software Architecture

The computational pipeline is partitioned strictly between time-critical deterministic tasks (STM32) and computationally intensive state estimation (Raspberry Pi Zero 2 W).





**Figure 12.** Distributed software architecture — high-level vision / navigation pipeline (Raspberry Pi) and the low-level deterministic control pipeline (STM32).

## 7.2 Low-Level Control Loops & Sensor Fusion (STM32 / C++)

The STM32 firmware is written in Arduino-core C++ using hardware abstraction layer (HAL) primitives configured for maximum deterministic execution speed:

- **High-Frequency Motor PID Loop (1 kHz):** executed inside the interrupt service routine (ISR) of hardware Timer 2. Quadrature encoder ticks from the N20 motors are accumulated via Timer 3 and Timer 4 in encoder interface mode. Velocity error is evaluated every 1.0 ms:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

Output values are clamped and written directly to Timer 1 PWM registers driving the TB6612FNG Hbridge pins.

- **Line Detection (QTR-8RC):** the 8-channel infrared array is polled using parallel digital pin discharge timing. A weighted centroid algorithm computes the line position offset relative to the vehicle centre:

$$\text{Position} = \frac{\sum_{i=0}^7 i \times S_i \times 1000}{\sum_{i=0}^7 S_i}$$

If the line vector diverges rapidly without a command from the Pi, the STM32 overrides high-level steering to prevent crossing boundary markers.

- **Safety Watchdog Timer:** a hardware Independent Watchdog (IWDG) is implemented alongside a communications watchdog. If a valid command packet from the Raspberry Pi Zero 2 W is not received within 100 milliseconds, the STM32 automatically cuts motor drive to zero, halting the vehicle safely.

### 7.3 High-Level Vision & Path Generation (Custom OpenCV Pipeline)

The Raspberry Pi Zero 2 W is designed to execute a dedicated computer-vision pipeline engineered for low latency:

- **Frame Ingestion:** video frames are captured using the hardware-accelerated libcamera backend, pulling 640×480 frames at 30 FPS to reduce memory bandwidth overhead.
- **Pre-Processing (OpenCV):** frames are converted to the HSV colour space. A dynamic threshold isolates the track boundaries and obstacles, filtering out background noise using Gaussian blur and morphological transformations (erosion / dilation).
- **Contour & Centroid Math:** OpenCV's findContours and moments functions calculate the centroid of the track lane. The difference between the centroid's X-coordinate and the centre of the camera frame provides the raw steering error.
- **Direct Serial Bridge:** a lightweight serial thread formats the calculated linear velocity ( $v$ ) and angular steering adjustment ( $\omega$ ) into a 4-byte payload, transmitted directly over the /dev/ttyAMA0 UART hardware pins to the STM32 at 115200 baud — eliminating the latency of network middleware.



**Figure 13.** Standalone OpenCV threshold pipeline (conceptual) — raw feed, HSV-masked binary feed, and the resulting steering-vector overlay.

## 7.4 Core Firmware Snippet (Dynamic Corner-Speed Control)

The full STM32 C++ firmware repository is available in Folder A: Source Code. The snippet below highlights the predictive kinematics engine. Because centrifugal cornering force is squared ( $F = m \times v^2 / R$ ), the STM32 actively recalculates a non-linear speed ceiling based on the Raspberry Pi's upcoming track-severity percentage, ensuring the rover never exceeds its mechanical grip limit entering a chicane.

```
// ===== //
DYNAMIC CORNER-SPEED CONTROL
// =====
// Cornering force:  $F = m * v^2 / R$ 
// Because speed is squared, the car must slow down
// non-linearly as the upcoming road curvature increases.

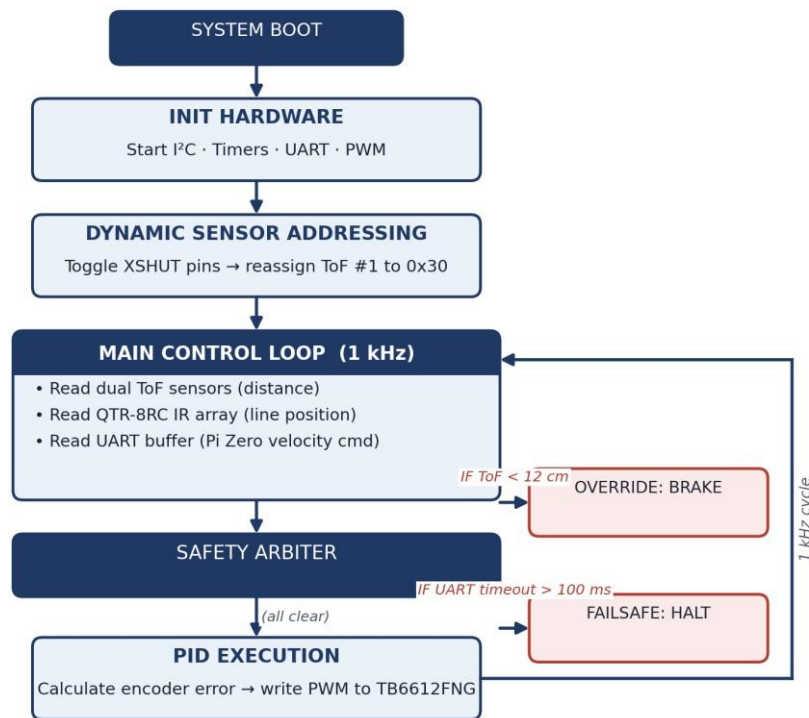
float getVisionCornerSpeedCapRPM()
{
    if (!visionIsTrusted())
    {
        return
VISION_FALLBACK_RPM;
    }

    // Normalize camera's upcoming severity prediction (0.0 to 1.0)
    float severity = constrain(
        (float)visionCornerSeverityPct / 100.0f,
        0.0f, 1.0f
    );

    // Apply non-linear physics limit:  $1 / \sqrt{1 + K * severity}$ 
    float speedFactor = 1.0f / sqrtf(
        1.0f + CORNER_CURVATURE_GAIN * severity
    );

    float capRPM = NORMAL_BASE_RPM * speedFactor;

    // Clamp the final target to safe mechanical limits
    return constrain(
        capRPM,
        VISION_MIN_BASE_RPM,
        NORMAL_BASE_RPM
    );
}
```



**Figure 14.** Low-level firmware state machine and closed-loop control flow, including the safety-arbiter override and failsafe paths.

## 8. Verification, Testing Protocols, & Race Preparation

### 8.1 Rigorous Subsystem Testing Protocol

Before track deployment, the hardware and firmware pipelines undergo strict bench testing:

1. **Thermal & Electrical Load Test:** run both N20 gearmotors at full forward-reverse oscillation under load for 10 minutes while recording thermal dissipation on the TB6612FNG driver and voltage stability across the LM2596 buck converter using the INA219 current sensor. Ensure no logic resets occur under motor stall conditions.
2. **Dynamic Range Verification:** move obstacle barriers at varying velocities across the dual VL53L1X sensor fields of view to confirm dynamic I<sup>2</sup>C address resolution operates without packet collisions or bus lockups.
3. **Failsafe Disconnect Validation:** engage high-speed motor rotation on the bench and physically depress the panel-mount E-Stop to verify instantaneous cutoff of all actuator lines. Disconnect the high-speed UART serial bridge during active drive to confirm the firmware watchdog cuts motor torque in under 100 ms.

### 8.2 Summary Conclusion

The APEX-1 design meets every technical and safety requirement established for the Robo Grand Prix Division at the architectural and specification level. By pairing a dual-tier ARM computing architecture with low-CoG, 3D-printed monocoque chassis and a locally sourced, cost-optimized electrical grid, the design balances high track speed with reliable obstacle evasion and calculated structural durability. Core control and safety logic has been exercised in simulation; physical bench



calibration, full Pi-to-STM32 UART integration, and on-track validation are the next phase ahead of the race qualification.